

Date: _____

Day: _____

RIPHAH INTERNATIONAL UNIVERSITY
Semester: 03

Data Structures
Assignment #01

Submitted To:

Dr. Sir Juncid Khan

Submitted By:

M. Rayyan [75531]

Date of Submission:

27/09/2026

Fintech Trading Management System

1. Project Description:

The Fintech Trading Management System is a java based program that manages financial assets & trading activities. It store assets information, searches and sorts assets, maintains a watchlist, calculates portfolio value, records transactions and processes trading orders.

2. Data Structure & Algorithms used:

1. **Arrays:** An array stores 10 financial assets including their ID, name, type, price & percentage change. It is also used to calculate the average, highest and lowest prices.

2. **Linear Search:** Searches for an array asset by checking its ID one by one until the required asset is found.

3. **Binary Search:** Searches for an asset by repeatedly checking the middle element. It uses low, high & mid. The asset IDs must be sorted.

4. **Bubble Sort:** Compares adjacent assets and swaps them if they are in the wrong price order.

5. **Selection Sort:** Finds the asset with the minimum price and places it at the correct position.

6. **Insertion Sort:** Takes an asset & inserts it into its correct position among the previously sorted assets.

7. **Merge Sort:** Divides the asset array into smaller parts, sorts them recursively and merges them into a sorted array.

8. **Quick Sort:** Selects the last asset as the pivot and places smaller priced assets before it and larger priced assets after it. It uses Lomuto partition.

9. **Singly Linked List:** Used for the watchlist. Each node stores an asset and a reference to the next node. It supports insertion, deletion, searching and display.

10. Recursion: Used to calculate the total portfolio value and display portfolio information. A recursive method calls itself until it reaches its base case.

11. Stack: Stores recent transactions using an array. It follows LIFO, so the latest transaction is removed first.

12. Queue: Stores trading orders using an array. It follows FIFO, so the earliest order is processed first.

3. Dry Runs:

1. Binary Search:

Task: Search for Asset ID: 105

Initial array of IDs:

[101, 102, 103, 104, 105, 106, 107, 108, 109, 110]

Step	Low	High	Mid	ID at Mid	Result
1	0	9	4	105	Found

Explanation: The middle index is $(0+9)/2 = 4$.

The asset at index 4 has id 105. So the search returns index 4.

Output is "Asset found: Ethereum"

Date: _____

Day: _____

Dry Run:

2. Bubble Sort:

Task: Sort three asset prices in ascending order.

Initial prices: [190, 175, 185]

Step	Comparison	Array after swap
1	$190 > 175 \rightarrow \text{swap}$	[175, 190, 185]
2	$190 > 185 \rightarrow \text{swap}$	[175, 185, 190]
3	$175 > 185 \rightarrow \text{no swap}$	[175, 185, 190]

Output: [175, 185, 190]

3. Recursive Portfolio Value:

Task: Calculate the portfolio value for these three assets.

Asset	Price	Qty	Value
Apple	190	2	380
Bitcoin	65000	1	65000
Gold	2300	3	6900

The recursive method calculates:

$$\text{total}(0) = 190 \times 2 + \text{total}(1)$$

$$\text{total}(1) = 65000 \times 1 + \text{total}(2)$$

$$\text{total}(2) = 2300 \times 3 + \text{total}(3)$$

$$\text{total}(3) = 0$$

Date: _____

Day: _____

returning from the recursive call:

$$\text{total}(2) = 6900 + 0 = 6900$$

$$\text{total}(1) = 65000 + 6900 = 71900$$

$$\text{total}(0) = 320 + 71900 = 72280$$

End Output: Total portfolio Value: 72280.0

Base Case: When $\text{index} == \text{assets.length}$, the method returns 0.

Recursive Case: The method multiplies the current asset's price by its quantity q_i and adds the value returned by the next recursive call.